EETS 7302 TCP/IP Network Administration

Session 12
Linux Admin – part3:

Scheduling jobs

- Crond and atd services can be used to schedule the tasks to be run at a certain time.
- Here atd service executes future tasks once only while the Crond service executes the tasks on recurring basis.
- Cron service is one of the form of automation
- crond service is responsible for execution of the tasks.

Scheduling jobs(cont'd)

- How the Cron service works?
  - We define tasks that are to be executed in the Cron configuration.
  - The Cron daemon looks for the tasks in the Cron configuration every minute to check if any tasks is to be executed.
  - Managing the Cron service is relatively easy
  - Whenever you make any configuration it doesn't need to be restarted or reloaded to activate the changes.
  - In many of the services in Linux you need to restart the service for the configuration changes to take place.

Scheduling jobs(cont'd)

- Cron service is started by default on Linux machine.
- Some of the basic tasks are already running such as log rotate i.e. a service that cleans up the log files and run the service.
- To see the status of Crond service is running use

  command: systemctl status crond –l

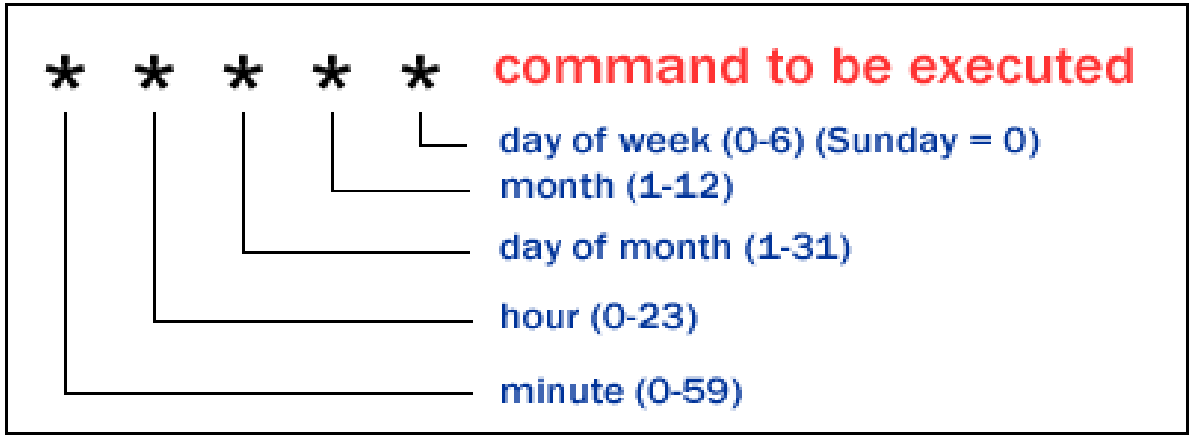
Scheduling jobs(cont'd)

- When scheduling the tasks through Cron you need to specify the timings so that tasks should be executed.
- It is a time string which we will specify in crontab configuration.
- Cron fields:

Field	Values
Minute	0-59
Day	0-23
Day of month	1-31
Month	1-12
Day of week	0-7

Scheduling jobs(cont'd)

- Note: In place of month and day of week we can specify the month string (ex. August) or day string (ex. Monday) but this is not preferable.
- Cron format:



Scheduling jobs(cont'd)

- If there is '*' in any place it can refer to any value.
- Example: 0 11 * * 1 /pathtoscript

  Run the tasks every Monday at 11:00
- We can also set the fractional value.

  Example: 0 */2 * 10 4 /pathtoscript

  Run the tasks every two hour on Thursday in the month of October.

Scheduling jobs(cont’d)

- The main configuration file for the Cron is /etc/crontab but it is not advisable to edit this file directly.
- Instead of modifying directly we can edit in one of these locations:
  - Cron files in /etc/cron.d.
  - Scripts in /etc/cron.hourly, cron.daily, cron.weekly, and cron.monthly.
  - User specific files that are created with crontab –e.

Scheduling jobs(cont’d)

- We can edit user specific Cron file using:
  - crontab –e
  - and specify the tasks we want to run and these jobs are then pushed to the Cron jobs that are to be run on the system.

Scheduling jobs(cont'd) : 5min. exercise

- Try it out:
  - See the current Cron jobs scheduled. Use cat /etc/crontab.
  - You will also get the overview of crontab.
  - Use crontab –e to schedule the job to update the system i.e yum update –y on every day in morning at 6 am.

    command: * 6 * * * yum update –y
  - 5 min. Exercise: Use crontab –e to schedule the job to record the date at every minute in a give file in your home directory

  - To view the crontab you have set use the command crontab -l

Scheduling jobs(cont'd)

- Try it out:
  - See the current Cron jobs scheduled. Use cat /etc/crontab.
  - You will also get the overview of crontab.
  - Use crontab –e to schedule the job to update the system i.e yum update –y on every day in morning at 6 am.

    command: * 6 * * * yum update –y
  - 5 min. Exercise: Use crontab –e to schedule the job to record the date at every minute in a file in your home directory

    command: * * * * * /bin/date >> /home/your_user_name/dates.output

  - To view the crontab you have set use the command crontab -l

Process management

- Process management is one of the key to efficiently manage the Linux server.
- As the process is started for each service, management of this is vital.
- The processes are of two types:
  - Shell jobs that are started from the command line also known as interactive process.
  - Daemons that provide services when we start the service or boot the server.

Process management (cont’d)

- Any command executed on terminal by default start the process in foreground(fg). This commands generally takes little time to complete.
- But whenever the command takes much time its advisable to start the process in background(bg). These might be the process that does not require user interaction.
- To start the process in background put ‘&’ behind it.

Process management (cont’d)

- To move the last foreground job in background use fg command. This command will move the last job back to foreground immediately.
- User ‘jobs’ command to get the overview of all current jobs. It might be useful when moving jobs between foreground and background.

Process management (cont’d)

- If a job is taking longer then predicted you may want to stop the job user Ctrl+z to temporarily stop the job.
  - But it does not remove job from the memory.
- To stop the job and also to remove it from the memory user Ctrl+c.

Process management (cont'd)

- Job Management overview

Command	Usage
Ctrl+Z	Stops the job temporarily so that it can be managed further.
Ctrl+D	Sends EOF character to the current job.
Ctrl+C	It cancels the current interactive job and it removes from the memory.
&(use at the end of command line)	To start the process in background
bg	Continue the job that has been stopped using Ctrl+Z in background

Process management (cont'd)

Command	Usage
fg	Bring the last job to foreground from background
jobs	Display which are the jobs running from a shell. Display job number that can be user as an argument to the commands bg and fg.

Process management (cont’d)

- When a process is started from the shell it becomes the child process of that shell.
- The parent is managing the child and is called as parent-child relationship.
  - So when a shell is terminated the process are also gone.
- But the process started in background will not be killed and we can kill those processes using different command

Process management (cont’d)

- When a process is started it can start multiple threads. So it is important to know to manage process and threads.
- Every process on Linux has unique process identification number called as PID.
- There are also some processes which are started by default by kernel and we cannot kill those process not change the priority of these process.
- To look for the process running by current user use ps command.

Process management (cont'd)

- To display all the process use command: ps aux or ps –ef.
- There are various extension for using ps command:

Option	Description
a	Display all the process.
T	Select all the process associated with this terminal
r	To show only the running process

Process management (cont'd)

- Output of all process:

```
[root@localhost ~]# ps aux
USER        PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root          1  0.0  0.8 128164  4440 ?        Ss   Nov12   0:25 /usr/lib/systemd/systemd
root          2  0.0  0.0      0     0 ?        S    Nov12   0:00 [kthreadd]
root          3  0.0  0.0      0     0 ?        S    Nov12   0:00 [ksoftirqd/0]
root          5  0.0  0.0      0     0 ?        S<   Nov12   0:00 [kworker/0:0H]
root          6  0.0  0.0      0     0 ?        S    Nov12   0:00 [kworker/u2:0]
root          7  0.0  0.0      0     0 ?        S    Nov12   0:00 [migration/0]
root          8  0.0  0.0      0     0 ?        S    Nov12   0:00 [rcu_bh]
root          9  0.0  0.0      0     0 ?        R    Nov12   0:04 [rcu_sched]
root         10  0.0  0.0      0     0 ?        S    Nov12   0:01 [watchdog/0]
root         12  0.0  0.0      0     0 ?        S    Nov12   0:00 [kdevtmpfs]
root         13  0.0  0.0      0     0 ?        S<   Nov12   0:00 [netns]
root         14  0.0  0.0      0     0 ?        S    Nov12   0:00 [khungtaskd]
root         15  0.0  0.0      0     0 ?        S<   Nov12   0:00 [writeback]
root         16  0.0  0.0      0     0 ?        S<   Nov12   0:00 [kintegrityd]
root         17  0.0  0.0      0     0 ?        S<   Nov12   0:00 [bioset]
root         18  0.0  0.0      0     0 ?        S<   Nov12   0:00 [kblockd]
root         19  0.0  0.0      0     0 ?        S<   Nov12   0:00 [md]
root         25  0.0  0.0      0     0 ?        S    Nov12   0:00 [kswapd0]
root         26  0.0  0.0      0     0 ?        SN   Nov12   0:00 [ksmd]
root         27  0.0  0.0      0     0 ?        S<   Nov12   0:00 [crypto]
```

Process management (cont'd)

- From the previous output you can see every process has PID. The root process has name in square brackets.
- The command ps aux is based on Linux system and ps –ef is for the unix system but it runs on most of the Linux system
- We can also view the process as the tree structure using command: pstree. Make sure to install it using command: yum install psmisc.

Process management (cont’d)

- To see the tree structure with parameters use –a option. Command: pstree –a
- To kill a process we can use command: kill PID. This is use to kill a single process and we need to specify the process id for that.
- The linux kernel allows many signals to be sent to processes to take the action.
- The kill command send the SIGTERM signal to process. The overview of the signals described in the table.

Process management (cont'd)

Signal	Value	Description
SIGHUP	1	To shutdown and restart the process
SIGINT	2	Interrupt the process
SIGQUIT	3	Quit from keyboard
SIGKILL	9	Kill the process
SIGTERM	15	Terminate the process
SIGSTOP	17,19,23	Stop process.

Process management (cont'd)

- To user the signal with kill command:

  kill –number PID

- For example to kill the process id here for httpd:

```
[root@localhost ~]# ps aux | grep httpd
root     23457  0.0  0.9 221948  4976 ?        Ss   07:30
apache   23490  0.0  0.5 221948  2960 ?        S    07:34
apache   23491  0.0  0.5 221948  2960 ?        S    07:34
apache   23492  0.0  0.5 221948  2960 ?        S    07:34
apache   23493  0.0  0.5 221948  2960 ?        S    07:34
apache   23494  0.0  0.5 221948  2960 ?        S    07:34
root     23515  0.0  0.1 112660   972 pts/0    R+   07:35
[root@localhost ~]# kill -9 23457
```

Process management (cont’d)

- In this example we have use signal SIGKILL to kill the process. After this the service has killed.
- To see the list of available signals user kill –l.
- To kill the multiple process we can use the command killall or pkill.
- Killall will kill the process name. For example, we can use killall process_name. Suppose we need to kill the process httpd we use killall httpd. This will terminate all the process simultaneously.

Process management (cont’d)

- While pkill we can specify name, PID, user etc.
- To kill all the process of apache user killall –u apache or if httpd killall –u httpd.
- Sometime pgrep can he helpful command to work with. It looks for the currently running processes and lists out the process IDs which match the selection criteria. There is many options available with pgrep which you can have a look on man page (i.e. man pgrep).

Process management (cont'd)

- Pgrep to list out process associated with apache.

```
[root@localhost ~]# pgrep -u apache
23601
23604
23605
23606
23607
23608
23609
```

- To get the overview of most active current process running use top command. From here we can perform command management tasks such as kill the process etc.

Process management (cont'd)

- Try it out:
  - Install httpd on machine.

    Use command: yum install httpd –y
  - To check installation use command: yum info http or yum list installed | grep httpd
  - To start the httpd service run service httpd start.
  - Now check the status using service httpd status. It should be in the running state.
  - Check the PID using ps aux | grep httpd.
  - Use pgrep httpd to see all the PID assigned to httpd.

Process management (cont’d)

- We can kill each process by specifying kill PID.
- Or use killall httpd to kill all the process associated with httpd. Here we are going to use killall httpd.
- Now check using ps aux | grep httpd and we can see all the processes has been killed.
- See the status of httpd using service httpd status.
- To restart the service use httpd service restart.

Free

- The free command displays system memory utilization.

- The Mem: row shows physical memory usage, while the Swap: row shows the usage of the system swap space, and the -/+ buffers/cache: row shows the measure of physical memory currently dedicated to system buffers.

```
[smu_user@localhost ~]$ free
              total        used        free      shared  buff/cache   available
Mem:        1883560      625596      528512       10176      729452     1059996
Swap:        839676           0      839676
```

vmstat

- For a more concise understanding of system performance, use vmstat.
- With vmstat, it is conceivable to get an overview of process, memory, swap, I/O, system, and CPU activity in one line of numbers:
  The principal line separates the fields in six classifications, including process, memory, swap, I/O, framework, and CPU related insights. The second line additionally recognizes the substance of each field, making it simple to rapidly check information for particular insights.

```
[smu_user@localhost ~]$ vmstat
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0      0 528252   2116 726316    0    0  2713   929  644 1525  9  5 83  3  0
[smu_user@localhost ~]$
```

- The process-related fields are:
  - r — The number of runnable processes waiting for access to the CPU
  - b — The number of processes in an uninterruptible sleep state
- The memory-related fields are:
  - swpd — The measure of virtual memory utilized
  - free — The measure of free memory
  - buff — The measure of memory utilized for buffers
  - cache — The measure of memory utilized as page reserve
- The swap-related fields are:
  - si — The measure of memory swapped in from disk
  - so — The measure of memory swapped out to disk
-

- The I/O-related fields are:
  - bi — Blocks sent to a block device
  - bo — Blocks received from a block device
- The system-related fields are:
  - in — The number of interrupts per second
  - cs — The number of context switches per second
- The CPU-related fields are:
  - us — The percentage of the time the CPU ran user-level code
  - sy — The percentage of the time the CPU ran system-level code
  - id — The percentage of the time the CPU was idle
  - wa — I/O wait

- At the point when vmstat keeps running with no alternatives, just a single line is shown. The line contains averages, calculated from the time the system was last booted.
- Be that as it may, most system administrators don't depend on the information in this line, as the time over which it was gathered changes.
- Rather, most administrators exploit vmstat's capacity to dully show asset use information at set interims.
- For instance, the command vmstat 1 shows one new line of use information consistently, while the command vmstat 1 10 shows one new line for each second, however just for the following ten seconds.

- **The Sysstat Suite of Resource Monitoring Tools**
  - While the past tools might be useful for increasing more understanding into system performance over brief time spans, they are of little use past giving a snapshot of system, asset use.
  - Moreover, there are parts of system execution that can't be effortlessly checked utilizing such oversimplified tools.
  - In this manner, a more refined tools is necessary. Sysstat is such a tool.

- Sysstat contains the following tools related to collecting I/O and CPU statistics:

tools	Description
iostat	Displays an overview of CPU utilization, along with I/O statistics for one or more disk drives.
mpstat	Shows more top to bottom CPU statistics.

- Sysstat likewise contains tools that gather system resource utilization data and make every day reports in light of that data. These tools are:

tools	Description
sadc	Known as the system activity data collector, sadc collects system resource utilization information and writes it to a file.
sar	Delivering reports from the files made by sadc, sar reports can be created intuitively or kept in touch with a file for more intensive examination.

- **iostat command**
  - The iostat command at its most fundamental gives an overview of CPU and disk I/O insights:

```
Linux 3.10.0-693.el7.x86_64 (localhost.localdomain)     12/03/2017      _x86_64_
(1 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.72    0.21    0.57    0.21    0.00   98.28

Device:            tps    kB_read/s    kB_wrtn/s    kB_read    kB_wrtn
sda              10.67       220.72        79.86     577223     208836
dm-0             10.38       207.59        79.00     542881     206585
dm-1              0.04         0.85         0.00       2228          0
```

  - Below the first line (which contains the framework's bit form and hostname, alongside the present date), iostat shows an outline of the sytem's average CPU use since the last reboot. The CPU usage report incorporates the following rates:

- Percentage of time spent in user mode (running applications, etc.)
- Percentage of time spent in user mode (for processes that have altered their scheduling priority using nice(2))
- Percentage of time spent in kernel mode
- Percentage of time spent idle

- Underneath the CPU use report is the device usage report. This report contains one line for every dynamic disk gadget on the system and incorporates the accompanying data:
  - The device specification, displayed as dev<*major-number*>-*sequence-number*, where <*major-number*> is the device's major number[1], and <*sequence-number*> is a sequence number starting at zero.
  - The number of transfers (or I/O operations) per second.

- The number of 512-byte blocks read per second.
- The number of 512-byte blocks written per second.
- The total number of 512-byte blocks read.
- The total number of 512-byte block written.

- **The mpstat command**
  - The mpstat command at first appears no different from the CPU utilization report produced by iostat:
  -

- **sar command**
  - The sar command produces system use reports in light of the information gathered by sadc.
  - As configured in Red Hat Enterprise Linux, sar is naturally run to process the files consequently gathered by sadc.
  - The report files are written to /var/log/sa/ and are named sar<dd>, where <dd> is the two-digit portrayals of the earlier day's two-digit date.
  - sar is typically keep running by the sa2 script. This script is occasionally summoned by cron by means of the record sysstat, which is situated in /etc/cron.d/. As a matter of course, cron runs sa2 once every day at 23:53, enabling it to deliver a report for the whole day's data.

- Uptime
  - uptime reveals to us to what extent the system has been running.
  - uptime gives a one-line show of the accompanying data:
    - The present time
    - to what extent the system has been running
    - what number of users are as of now logged on
    - the system load averages for as long as 1, 5, and 15 minutes.

    - Uptime syntax: uptime [options]

options	description
_h, __help	Display a brief help message, and exit.
-V, --version	Display version information, and exit.

- Linux w command
  - The **w** command is a quick way to see who is logged on and what they are doing.
  - The of header he output shows (in this order): the current time, how long the system has been running, how many users are currently logged on, and the system load averages for the past 1, 5, and 15 minutes.
  - ‘w’syntax = w [*options*] *user* [...]

options	description
-h, **--no-header**	Don't print the header.
-u, **--no-current**	Ignores the username while figuring out the current process and cpu times. (To see an example of this, switch to the root user with "**su**" and then run both "**w**" and "**w -u**".)
-s, **--short**	Display abbreviated output (don't print the login time, JCPU or PCPU times).
-V, **--version**	Display version information, and exit.
user	Show information about the specified the *user* only.

End of session